

day12笔记

一、完整开发

1、前端开发 (html、css、JavaScript、vue.js、vant4)

2、后端开发 (python基础、FastAPI)

3、后端开发框架

- 第一个: <https://www.django.cn/>, 大而全 (能做的事情非常多, 对新手来说不是那么友好) --- Java
- 第二个: <https://fastapi.org.cn/>, 快速、便捷、简单 (容易上手)

二、FastAPI介绍

1、FastAPI是一个基于python的后端框架

2、设计接口、连接数据库

3、和前端进行通信 (http协议 --- 超文本传输协议)

4、api接口

- 应用程序编程接口 (Application Programming Interface, API), 也称应用编程接口, 是一组为不同软件组件之间交互而创建的**定义和协议**。API定义了软件模块之间如何相互交互, 包括可以调用哪些功能、如何调用、需要提供的**数据格式**, 以及返回结果的形式。通过API, 开发者可以复用现有能力, 而无需了解或修改底层实现细节。
- 理解: 其实就是一个地址 (有规则的), 可以通过地址进行前后端交互
- 前端 通过 接口 拿到后端数据
- 后端 通过 接口 拿到前端传递的数据

三、FastAPI使用

1、安装

```
pip install fastapi
```

注意点: 必须先切换镜像源地址, 不然默认在国外, 下载可能会出错, 下载到一半卡住

2、使用

```
# 导包
from fastapi import FastAPI

# 创建应用
app = FastAPI()

# 编写接口
"""
api接口类型
+ get 获取数据 --- 网页搜索、免费信息网站
```

```

+ post 提交数据 --- 登录、注册
+ put 替换数据 --- 修改密码、修改其他信息
+ delete 删除数据 --- 删除信息
+ all 不区分类型的，很多免费的接口都是使用这个来做的
+ ...
"""

# get('/') 括号里面定义的是接口地址路径（自定义）
# / 表示根路径（http://localhost、http:127.0.0.1）
@app.get('/')
def test_demo():
    return 'hello, 这是后端给你返回的数据！'

```

3、启动后端服务器

安装 uvicorn

```
pip install uvicorn
```

启动

```

# main 文件名称，不需要加后缀
# 注意点：如果启动过程中发现端口号被占用，需要手动设置端口号
uvicorn main:app

# 指定端口号
uvicorn main:app --port 8080

# 热更新启动
uvicorn main:app --reload --port 8080

# 指定地址 0.0.0.0 表示通过本机的ip地址来访问 192.168.224.163
# 注意点：地址指定要符合规则，不能像域名那样自定义
uvicorn main:app --host 0.0.0.0 --port 8080 --reload

```

注意点：在没有指定地址的时候，默认本地访问地址是：`http://localhost:端口号` 或者 <http://127.0.0.1:端口号>

快捷启动

```

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

四、接口测试方式

1、第一种方案，使用测试工具 `postman`，推荐可以使用浏览器插件来实现这个功能

2、第二种方案，使用 FastAPI 自带测试工具

3、第三种方案，就使用代码方式，自己写前端代码

4、注意点：

- 不使用 `postman` 工具，也可以直接在浏览器上地址栏进行测试

- 浏览器地址栏只支持 get请求
- `postman` 接口请求是没有问题的，如果你post请求携带参数，工具自身存在bug，携带不过去。这个时候就建议使用 FastAPI自带测试工具

5、大家发现使用工具优势，可以不写代码，快速知道这个接口是否存在问题

五、路由router

1、路由是URL地址和处理函数之间的映射（一一对应）关系，它决定了当用户访问某个特定网址时，服务器执行对应的代码返回结果

2、router其实就是对API接口的管理

```
@app.get()

@app.post()

@app.put()

@app.delete()

...
```

3、路由地址

- 路由地址是api接口的关键，因为一个地址对应一个功能
- 路由地址可以自定义，建议自定义的时候最好语义化

```
# 定义新闻列表接口
@app.get('/dzy/news/games/list')
def game_list():
    return '新闻列表，游戏模块'
```

六、请求参数

1、是url地址的一部分，可以通过参数携带数据

`http://127.0.0.1/dzy/news/games/list?a=10&b=20`

2、参数类型

- 查询参数
- json数据
- 表单数据

3、示例代码

```
# 导包
import uvicorn
from fastapi import FastAPI

# 创建应用
```

```

app = FastAPI()

# 编写接口
@app.get('/')
def test_demo():
    return '欢迎来到服务器!'

# 携带参数的接口
# 注意点: 查询参数传递过来的数据类型是字符串
@app.get('/shop/{product_id}')
def shop(product_id):
    if product_id == '100':
        return f'这是A商品, id编号: {product_id}'
    elif product_id == '200':
        return f'这是B商品, id编号: {product_id}'
    else:
        return f'这是商品接口, {product_id}'

# 根据多个参数返回不同的数据
fake_item_db = [
    {"id": 0, "name": "商品A"},
    {"id": 1, "name": "商品B"},
    {"id": 2, "name": "商品C"},
    {"id": 3, "name": "商品D"},
    {"id": 4, "name": "商品E"}
]
@app.get('/pages/data')
# skip 表示跳过几条数据
# limit 表示返回最大数据是多少
def data_page(skip: int = 0, limit: int = 10):
    # 1: 1 + 3 = 4
    return fake_item_db[skip: skip + limit] # [1:4]

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

4、json数据传递

```

# 导包
import uvicorn
from fastapi import FastAPI

# 创建应用
app = FastAPI()

# 编写接口
@app.get('/')
def test_demo():
    return '欢迎来到服务器!'

# 注册接口
@app.post('/register')
def register_fn(username: str, password: str):

```

```

# 第一步：读取文件内容，进行检测查看用户名是否存在，如果存在就给前端返回此用户名太受欢迎请更好一个
# 第二步：把拿到的数据进行写入文件操作
# 第三步：进行存储，给前端返回注册成功
print(username, password)
return '这是注册接口'

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

5、表单数据传递

安装

```
pip install python-multipart
```

示例代码

```

# 导包
import uvicorn
from fastapi import FastAPI, Form

# 创建应用
app = FastAPI()

# 编写接口
@app.get('/')
def test_demo():
    return '欢迎来到服务器!'

@app.post('/register')
def register_fn(username: str=Form(...), password: str=Form(...)):
    print(username, password)
    return '这是注册接口'

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

七、数据响应

- 1、后端可以给前端返回各种类型的数据，默认可以返回字符串，但是其他的数据响应需要对应的FastAPI方法支撑
- 2、字符串和json数据

```

@app.get('/demo1')
def demo1():
    return '这是字符串数据'

@app.get('/demo2')
def demo2(user: str):
    # return [{"name": "张三", "age": 18}]
    if user == 'admin':
        return {"code": 1, "msg": '登录成功'}
    else:
        return {"code": 0, "msg": '用户名或者密码错误'}

```

3、响应HTML数据

```

# 导包
import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

# 创建应用
app = FastAPI()

# 编写接口
@app.get('/')
def test_demo():
    return '欢迎来到服务器!'

@app.get('/demo3', response_class=HTMLResponse)
def demo3():
    return '<h1>hello 美女帅哥! </h1>'

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

八、重定向

1、你访问的是A地址，重定向到B地址

2、示例代码

```

# 导包
import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse, RedirectResponse

# 创建应用
app = FastAPI()

# 编写接口
@app.get('/')
def test_demo():
    return '欢迎来到服务器!'

```

```

@app.get('/dzy')
def dzy():
    return "欢迎来到丁致字真爱有缘网"

@app.get('/item')
def item():
    return RedirectResponse(url='/dzy')

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)

```

3、真实场景

```

@app.get('/home')
def home():
    return '欢迎来到APP首页'

@app.get('/')
def root_fn():
    return RedirectResponse(url='/home')

```

九、中间件

- 1、中间件指的是发送请求 和 响应请求中间的环节
- 2、中间件解决跨越问题
- 3、示例代码

```

# 导包
import uvicorn
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

# 创建应用
app = FastAPI()

# 解决跨越问题
app.add_middleware(
    CORSMiddleware, # 使用这个中间件 * 表示所有
    allow_origins=['*'], # 允许所有来源
    allow_methods=['*'], # 允许请求方式
    allow_headers=['*'], # 允许前端携带那些请求头信息 例如: token
    allow_credentials=True, # 表示是否允许前端携带被认证的cookie信息
)

@app.get('/home')
def home():
    return '首页数据'

@app.get('/category')

```

```
def category():
    return '分类页面数据'

if __name__ == '__main__':
    uvicorn.run('app:app', host='127.0.0.1', port=8080, reload=True)
```

十、前端发送请求

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h1>使用代码方式请求后端服务器数据</h1>
    <hr>
    <button>request</button>
    <script>
        /*
            跨域规则 --- 其中两个地址之间有一个不相同就说明跨越了
            + 协议
            + 域名
            + 端口号

            发送请求的地址（前端）
            +
            http://localhost:63342/AI_project/day12/code/%E5%8F%91%E9%80%81%E8%AF%B7%E6%B1%82.html?
            _ijt=6lnif
            后端返回响应信息的地址（后端）
            + http://127.0.0.1:8080/category
        */
        const btn = document.querySelector('button')
        btn.onclick = ()=>{
            // 发送请求 ajax
            // 实例化 xhr对象 --- 通过 xhr对象来发送请求的
            const xhr = new XMLHttpRequest()
            // 填写参数信息
            // 参数1表示请求类型（根据后端接口来设置）
            // 参数2表示请求接口地址
            xhr.open('get', 'http://127.0.0.1:8080/home?a=10&b=20')
            // 监听请求状态
            xhr.onload = ()=>{
                console.log(xhr.responseText)
            }
            // 发送请求
            xhr.send()
        }
    </script>
</body>
</html>
```

